

Práctica 4 — Balanceador de carga Apache

Asignatura: Sistemas Informáticos II

Grupo / Pareja: Alejandro Pardo, Pablo Pollo

Fecha: 02 de mayo de 2026

Introducción

El objetivo de esta práctica es desplegar un clúster de tres instancias de la aplicación Django visaApp (gestor de pagos electrónicos) y configurar un balanceador de carga Apache (mod_proxy_balancer) delante del clúster, usando *sticky sessions* sobre la cookie ROUTEID. Sobre este montaje se ejecutan ocho ejercicios que estudian la afinidad de sesión, el reparto de carga, los escenarios de fallo (failover) y de recuperación (failback), y se finaliza con una prueba de carga con JMeter.

Arquitectura desplegada

Elemento	VM	IP interna	Puerto
PostgreSQL (si2db)	VM1	192.168.56.11	5432
Apache (balanceador)	VM1	192.168.56.11	18080
Django Instance01	VM1	192.168.56.11	18000
Django Instance02	VM2	192.168.56.12	28000
Django Instance03	VM3	192.168.56.13	38000

Las tres VMs están en VirtualBox con dos adaptadores: NAT (para Internet y SSH al host) y una *Internal Network* si2net (red 192.168.56.0/24) que les permite verse entre sí. El acceso a la aplicación desde el navegador se hace en `http://192.168.56.11:18080/P1base/visaApp/tarjeta/` (con un *port-forward* localhost:8080 → 192.168.56.11:18080 cuando se accede desde el host).

Modificaciones realizadas en el código Django

Para poder identificar qué instancia ha atendido cada petición se han añadido los siguientes cambios al código de si2_alumnos-main/P1-base/:

1. **visaApp/models.py:** campo instancia = models.CharField(max_length=24, default='None') en el modelo Pago.

2. **visaApp/views.py**: en las vistas `aportarinfo_pago` y `testbd` se captura la cookie de afinidad:

```
pago_data['instancia'] = request.COOKIES.get('ROUTEID', 'None')
```
 3. **visaApp/templates/template_exito.html**: muestra `{{ pago.instancia }}` en la pantalla de pago realizado.
 4. **visaApp/templates/template_get_pagos_result.html**: añade columna *Instancia* a la tabla del listado.
 5. **visaSite/settings.py**:
 - `ALLOWED_HOSTS = ['*']` (necesario porque Apache reescribe el Host).
 - `SESSION_ENGINE = "django.contrib.sessions.backends.cache"` con `LocMemCache` (las sesiones quedan en memoria local de cada instancia, **no compartidas**).
-

Ejercicio 1 — Configuración inicial y comprobación

Enunciado: Verificar que el clúster está operativo: Apache activo, configuración válida y los tres BalancerMember en estado OK desde el balancer-manager.

Estado de Apache

Comando: `sudo systemctl status apache2`

```
si2@si2-ubuntu-vm-1:~/P1base$ sudo systemctl status apache2 --no-pager
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-04-28 20:07:30 CEST; 45min ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 2094 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
 Main PID: 2098 (apache2)
    Tasks: 55 (limit: 1670)
   Memory: 7.7M (peak: 8.0M)
      CPU: 341ms
   CGroup: /system.slice/apache2.service
           └─2098 /usr/sbin/apache2 -k start
             └─2101 /usr/sbin/apache2 -k start
               └─2102 /usr/sbin/apache2 -k start

abr 28 20:07:30 si2-ubuntu-vm-1 systemd[1]: Starting apache2.service - The A....
abr 28 20:07:30 si2-ubuntu-vm-1 apachectl[2097]: AH00558: apache2: Could not...age
abr 28 20:07:30 si2-ubuntu-vm-1 systemd[1]: Started apache2.service - The Ap...er.
Hint: Some lines were ellipsized, use -l to show in full.
```

Validación de la configuración

Comando: `sudo apachectl configtest`

Resultado: Syntax OK.

```
si2@si2-ubuntu-vm-1:~/P1base$ sudo apachectl configtest
sudo apachectl -S 2>&1 | head -20
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1. Set the 'ServerName' directive globally to suppress this message
Syntax OK
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1. Set the 'ServerName' directive globally to suppress this message
VirtualHost configuration:
*:18080                 192.168.56.11 (/etc/apache2/sites-enabled/000-default.conf:1)
ServerRoot: "/etc/apache2"
Main DocumentRoot: "/var/www/html"
Main ErrorLog: "/var/log/apache2/error.log"
Mutex watchdog-callback: using_defaults
Mutex proxy-balancer-shm: using_defaults
Mutex rewrite-map: using_defaults
Mutex proxy: using_defaults
Mutex default: dir="/var/run/apache2/" mechanism=default
PidFile: "/var/run/apache2/apache2.pid"
Define: DUMP_VHOSTS
Define: DUMP_RUN_CFG
User: name="www-data" id=33
Group: name="www-data" id=33
si2@si2-ubuntu-vm-1:~/P1base$
```

Balancer-manager con las 3 instancias

URL: `http://192.168.56.11:18080/balancer-manager`

Las tres rutas Instance01, Instance02 e Instance03 aparecen en estado *Init Ok* y con factor de carga 1.

← → ↻ ⓘ localhost:8080/balancer-manager

Escuela Politéc... Laboratorio Vi... Moodle UAM |...

Load Balancer Manager for localhost

Server Version: Apache/2.4.58 (Ubuntu)
Server Built: 2025-08-11T11:10:09
Balancer changes will NOT be persisted on restart.
Balancers are inherited from main server.
ProxyPass settings are inherited from main server.

LoadBalancer Status for [balancer://si2cluster](#) [p19a2b80f_si2cluster]

MaxMembers	StickySession	DisableFailover	Timeout	FailoverAttempts	Method	Path	Active
3 [3 Used]	ROUTEID	Off	0	2	byrequests	/P1base	Yes

Worker URL	Route	RouteRedir	Factor	Set	Status	Elected	Busy	Load	To	From
http://192.168.56.11:18000	Instance01		1.00	0	Init Ok	7	0	200	5.0K	6.0K
http://192.168.56.12:28000	Instance02		1.00	0	Init Ok	6	0	500	4.5K	2.5K
http://192.168.56.13:38000	Instance03		1.00	0	Init Ok	10	0	-700	6.6K	17K

Apache/2.4.58 (Ubuntu) Server at localhost Port 8080

Ejercicio 2 — Comportamiento sin sticky con curl

Enunciado: Apagar VM2 y VM3, hacer 3 peticiones con curl desde el host hacia el balanceador y observar el comportamiento. Después repetir desde el navegador y razonar las diferencias.

Apagado de VM2 y VM3

En cada VM: sudo poweroff.

```
si2@si2-ubuntu-vm-1:~/P1base$ # Confirma que VM2 y VM3 ya no responden (debe dar timeout)
curl --max-time 3 -s -o /dev/null -w "VM2: %{http_code}\n" http://192.168.56.12:28000/visaApp/tarjeta/
curl --max-time 3 -s -o /dev/null -w "VM3: %{http_code}\n" http://192.168.56.13:38000/visaApp/tarjeta/

# Reiniciar Apache para resetear estado del balanceador
sudo systemctl restart apache2
sudo systemctl is-active apache2
VM2: 000
VM3: 000
active
```

← → ↻ localhost:8080/balancer-manager

Escuela Politéc... Laboratorio Vi... Moodle UAM |...

Load Balancer Manager for localhost

Server Version: Apache/2.4.58 (Ubuntu)
 Server Built: 2025-08-11T11:10:09
 Balancer changes will NOT be persisted on restart.
 Balancers are inherited from main server.
 ProxyPass settings are inherited from main server.

LoadBalancer Status for [balancer://si2cluster](#) [p19a2b80f_si2cluster]

MaxMembers	StickySession	DisableFallover	Timeout	FalloverAttempts	Method	Path	Active
3 [3 Used]	ROUTEID	Off	0	2	byrequests/P1base		Yes

Worker URL	Route	RouteRedir	Factor	Set	Status	Elected	Busy	Load	To	From
http://192.168.56.11:18000	Instance01		1.00	0	Init Ok	15	0	-1100 11K	13K	
http://192.168.56.12:28000	Instance02		1.00	0	Init Ok	1	0	1200 0	0	
http://192.168.56.13:38000	Instance03		1.00	0	Init Err	2	0	-100 0	0	

Apache/2.4.58 (Ubuntu) Server at localhost Port 8080

Peticiones con curl y estado del balanceador

Comando ejecutado tres veces:

curl -v <http://192.168.56.11:18080/P1base/visaApp/tarjeta/>

```
si2@si2-ubuntu-vm-1:~/P1base$ # Petición 1
curl -s -I http://127.0.0.1:18080/P1base/visaApp/tarjeta/ | grep -iE 'HTTP|ROUTE ID'
HTTP/1.1 200 OK
Set-Cookie: ROUTEID=.Instance01; path=/
si2@si2-ubuntu-vm-1:~/P1base$
```

```
si2@si2-ubuntu-vm-1:~/P1base$ # Petición 2
curl -s -I http://127.0.0.1:18080/P1base/visaApp/tarjeta/ | grep -iE 'HTTP|ROUTE ID'
HTTP/1.1 200 OK
Set-Cookie: ROUTEID=.Instance01; path=/
si2@si2-ubuntu-vm-1:~/P1base$
```

```
si2@si2-ubuntu-vm-1:~/P1base$ # Petición 3
curl -s -I http://127.0.0.1:18080/P1base/visaApp/tarjeta/ | grep -iE 'HTTP|ROUTE ID'
HTTP/1.1 200 OK
Set-Cookie: ROUTEID=.Instance01; path=/
si2@si2-ubuntu-vm-1:~/P1base$
```

Las tres invocaciones devuelven HTTP 200 con el HTML del formulario en la respuesta. Lo relevante es el efecto que tienen en el balancer-manager.

Antes de la primera petición, los tres BalancerMember están en estado Init Ok con Elected = 0. Tras la primera invocación, Apache aplica round-robin (no hay cookie ROUTEID en curl) y elige una de las tres rutas; si toca Instance02 o Instance03 la conexión TCP falla porque las VM están apagadas, mod_proxy_balancer marca esa instancia como Init Err y reintenta la siguiente, hasta servir la petición desde Instance01.

Tras las tres invocaciones, el balancer-manager queda con Instance01 atendiendo todas las peticiones efectivas (Elected = 3) y las dos instancias caídas marcadas con estado Init Err.

Mismas peticiones desde el navegador

Las tres recargas del navegador se sirven todas desde Instance01. La razón es la cookie de afinidad: la primera respuesta del balanceador incluye Set-Cookie: ROUTEID=.Instance01, y el navegador la persiste y la reenvía en las dos siguientes peticiones. Apache, al detectar la cookie, respeta la afinidad y enruta directamente a Instance01 sin probar las otras rutas (que siguen marcadas Err).

La diferencia con curl es estructural: curl no comparte estado HTTP entre invocaciones (cada llamada es un proceso nuevo y no hereda cookies), mientras que el navegador sí mantiene un almacén de cookies por origen. Por tanto, sin sticky, curl provoca round-robin puro; con navegador, sticky garantiza que un mismo cliente siempre cae en la misma instancia.

Análisis

Las llamadas con curl no envían cookies entre invocaciones (cada curl es un proceso nuevo sin almacén persistente), por lo que el balanceador no encuentra ROUTEID y aplica lbmethod=byrequests repartiendo las peticiones en *round-robin*; al estar VM2 y VM3 apagadas, todas las peticiones acaban siendo servidas por Instance01, pero los contadores Elected de Instance02 e Instance03 aumentan inicialmente (intentos previos al marcado *Err*) hasta que el balanceador los marca como caídos.

Desde el navegador, en cambio, la primera respuesta lleva el Set-Cookie: ROUTEID=.Instance0X y todas las peticiones siguientes incluyen esa cookie, por lo que el balanceador respeta la afinidad y siempre va a la misma instancia (en este caso Instance01, la única viva). Esa es la diferencia esencial: curl no preserva estado HTTP entre llamadas, el navegador sí.

La conclusión práctica del ejercicio es doble: por un lado, sin afinidad de sesión, mod_proxy_balancer garantiza la disponibilidad gracias a la detección de fallos (los miembros caídos se marcan Err y dejan de recibir tráfico); por otro lado, el balanceo round-robin sólo es completo cuando el cliente no aporta cookies, lo que en un entorno real sólo ocurre con clientes como curl, monitores o herramientas de carga. En navegadores convencionales la cookie ROUTEID hace de pseudo-sesión y mantiene a cada usuario en la misma instancia, comportamiento que resulta crítico en los ejercicios siguientes.

Ejercicio 3 — Deshabilitar instancias desde el balancer-manager

Enunciado: Encender de nuevo las 3 VMs. En el balancer-manager deshabilitar dos instancias y registrar un pago. Comprobar en la BD qué instancia lo atendió.

Deshabilitación de Instance02 e Instance03

Desde el balancer-manager se modifican esas dos rutas marcando *Status: Disabled* y aplicando.

localhost:8080/balancer-manager?b=si2cluster&w=http://192.168.56.12:28000&nonce=5afe8f71-8774-5865-cba8-c4691b172fe5

Escuela Politéc... Laboratorio Vi... Moodle UAM |...

Load Balancer Manager for localhost

Server Version: Apache/2.4.58 (Ubuntu)
Server Built: 2025-08-11T11:10:09
Balancer changes will NOT be persisted on restart.
Balancers are inherited from main server.
ProxyPass settings are inherited from main server.

LoadBalancer Status for [balancer://si2cluster](#) [p19a2b80f_si2cluster]

MaxMembers	StickySession	DisableFailover	Timeout	FailoverAttempts	Method	Path	Active
3 [3 Used]	ROUTEID	Off	0	2	byrequests	/P1base	Yes

Worker URL	Route	RouteRedir	Factor	Set	Status	Elected	Busy	Load	To	From
http://192.168.56.11:18000	Instance01		1.00	0	Init Ok	0	0	100	0	0
http://192.168.56.12:28000	Instance02		1.00	0	Init Dis	0	0	100	0	0
http://192.168.56.13:38000	Instance03		1.00	0	Init Dis	0	0	100	0	0

Edit worker settings for [http://192.168.56.12:28000](#)

Load factor: 1.00

LB Set: 0

Route: Instance02

Route Redirect:

Status:

Ignore Errors

Draining Mode

Disabled

Hot Standby

Hot Spare

Stopped

On ☐

Off ☒

On ☐

Off ☒

On ☒

Off ☐

On ☐

Off ☒

On ☐

Off ☒

On ☐

Off ☒

Submit

Apache/2.4.58 (Ubuntu) Server at localhost Port 8080

Pago realizado desde el navegador

← → ↻ localhost:8080/P1base/visaApp/tarjeta/

Escuela Politéc... Laboratorio Vi... Moodle UAM |...

Introduzca la Informacion de la Tarjeta de la Persona que Paga (visaSite)

Número de Tarjeta:

Nombre:

Fecha de Caducidad:

Código de Autorización:

← → ↻ localhost:8080/P1base/visaApp/pago/

Escuela Politéc... Laboratorio Vi... Moodle UAM |...

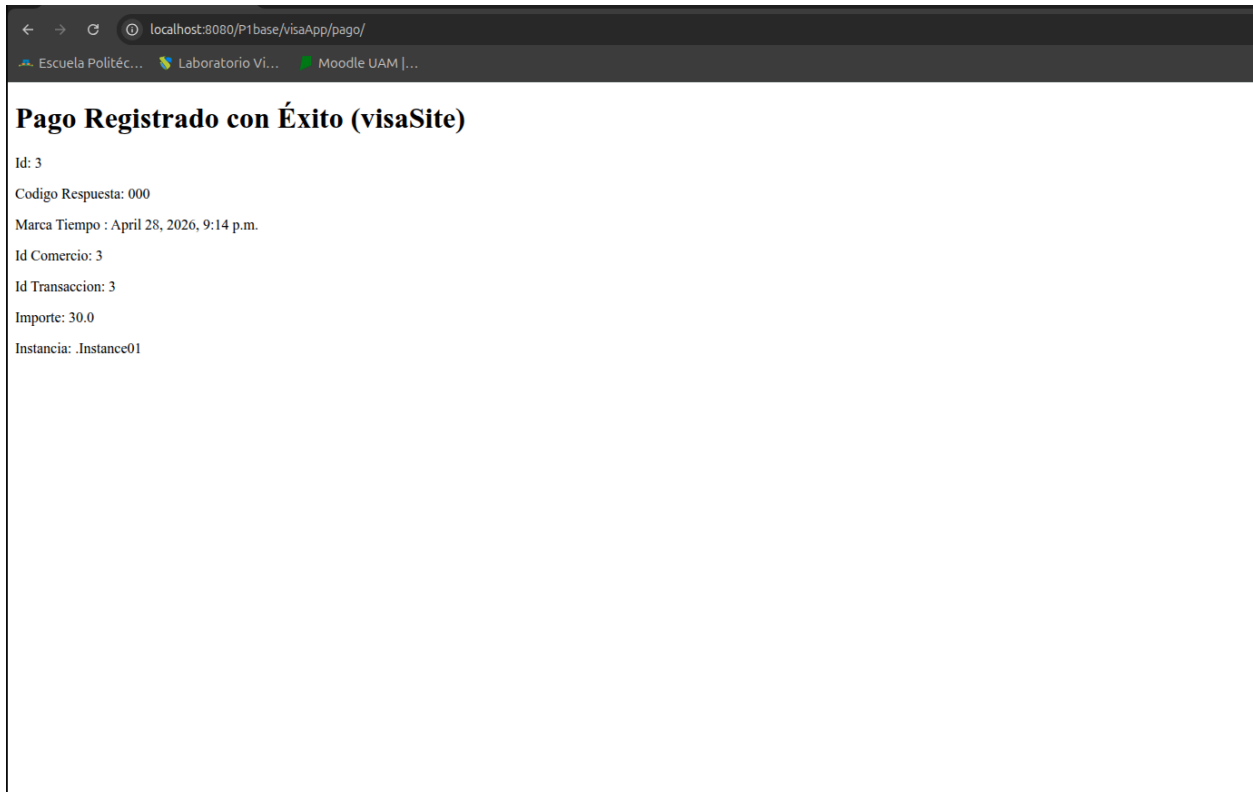
Registro de Pago (visaSite)

Introduzca los datos del nuevo pago a registrar:

ID Comercio:

ID Transaccion:

Importe:



Verificación en base de datos

```
si2@si2-ubuntu-vm-1:~/P1base$ sudo -u postgres psql -d si2db -P pager=off -c 'SELECT id, "idComercio", "idTransaccion", importe, instancia, "marcaTiempo" FROM pago ORDER BY id DESC LIMIT 5;'
 id | idComercio | idTransaccion | importe | instancia |          marcaTiempo
-----+-----+-----+-----+-----+-----
  3 | 3          | 3            | 30      | .Instance01 | 2026-04-28 21:14:51.058513+02
  2 | 2          | 2            | 20      | .Instance02 | 2026-04-28 20:49:55.011539+02
(2 rows)

si2@si2-ubuntu-vm-1:~/P1base$
```

El último registro tiene instancia = .Instance01, la única no deshabilitada, lo que confirma que el balanceador respeta el flag Disabled y enruta exclusivamente a las instancias activas.

Comparado con apagar las VM (Ejercicio 2), deshabilitar desde el balancer-manager es una operación administrativa en caliente: no se interrumpe el servicio, no requiere reiniciar Apache, y se aplica de manera inmediata. Es la herramienta natural para sacar una instancia del pool durante despliegues, mantenimiento o investigación de incidencias sin pérdida de tráfico.

Ejercicio 4 — Quitar sticky session

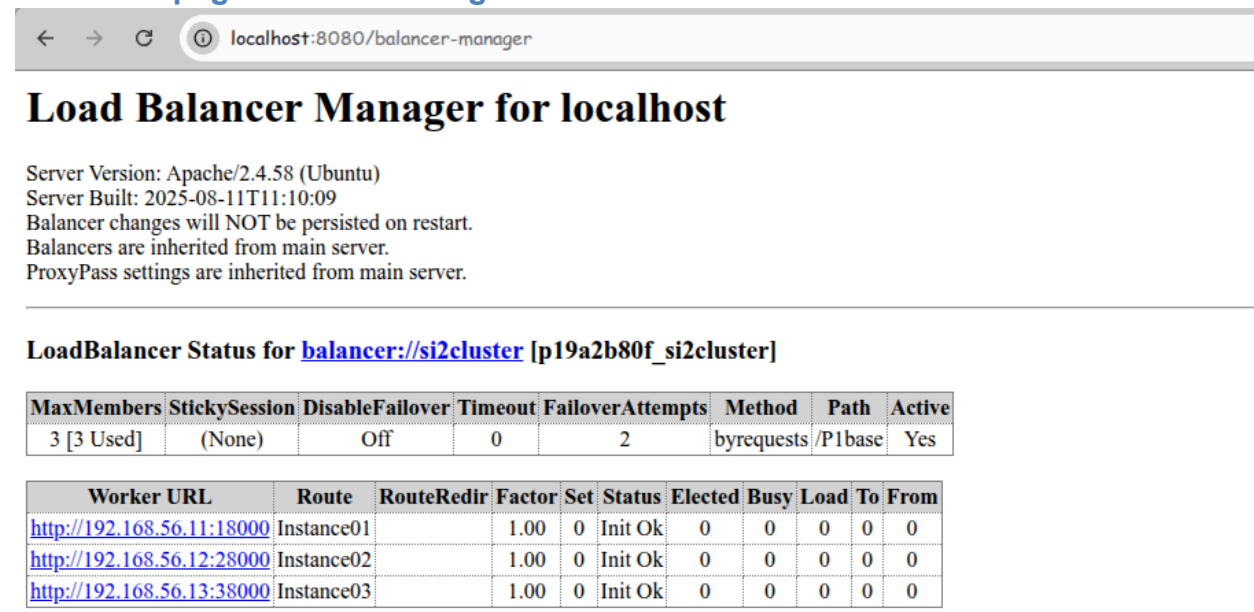
Enunciado: Volver a habilitar las tres instancias, quitar sticky session=ROUTEID y observar qué ocurre al intentar registrar un pago.

Balancer-manager sin sticky session

Tras editar 000-default.conf y reiniciar Apache, en balancer-manager aparece *Sticky Session: (None)*.

Tras editar /etc/apache2/sites-available/000-default.conf y eliminar la directiva ProxySet sticky session=ROUTEID, al reiniciar Apache y abrir balancer-manager se observa que el campo Sticky Session pasa de mostrar ROUTEID a mostrar (None). El balanceador deja por tanto de respetar la afinidad de cookie y volverá a aplicar lbmethod=byrequests en cada petición de manera independiente.

Intento de pago desde el navegador



Server Version: Apache/2.4.58 (Ubuntu)
Server Built: 2025-08-11T11:10:09
Balancer changes will NOT be persisted on restart.
Balancers are inherited from main server.
ProxyPass settings are inherited from main server.

LoadBalancer Status for [balancer://si2cluster](#) [p19a2b80f_si2cluster]

MaxMembers	StickySession	DisableFailover	Timeout	FailoverAttempts	Method	Path	Active
3 [3 Used]	(None)	Off	0	2	byrequests	/P1base	Yes

Worker URL	Route	RouteRedir	Factor	Set	Status	Elected	Busy	Load	To	From
http://192.168.56.11:18000	Instance01		1.00	0	Init Ok	0	0	0	0	0
http://192.168.56.12:28000	Instance02		1.00	0	Init Ok	0	0	0	0	0
http://192.168.56.13:38000	Instance03		1.00	0	Init Ok	0	0	0	0	0

Apache/2.4.58 (Ubuntu) Server at localhost Port 8080

Al rellenar la tarjeta y enviar el formulario, la aplicación responde con el mensaje:

¡Error: numero tarjeta no encontrado en la sesión!

▼ Aportar Informacion Tarje x +

← → ↻ ⓘ localhost:8080/P1base/visaApp/tarjeta/ ☆ Incog

Introduzca la Informacion de la Tarjeta de la Persona que Paga (visaSite)

Número de Tarjeta:

Nombre:

Fecha de Caducidad:

Código de Autorización:

▼ Registrar Pago (visaSite) x +

← → ↻ ⓘ localhost:8080/P1base/visaApp/pago/ ☆ Incog

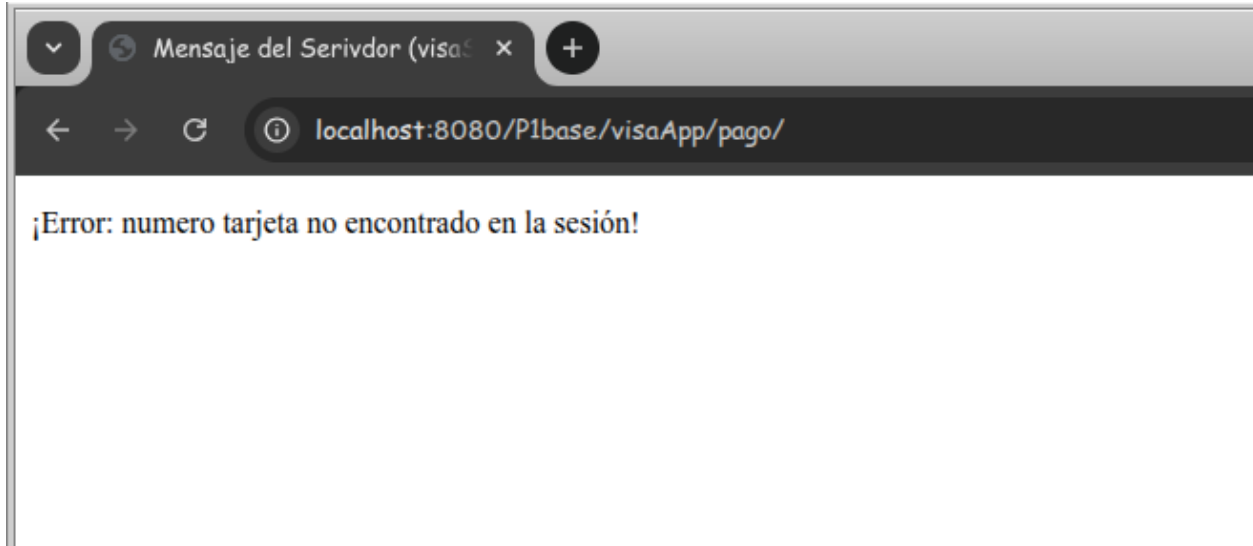
Registro de Pago (visaSite)

Introduzca los datos del nuevo pago a registrar:

ID Comercio:

ID Transaccion:

Importe:



Análisis

Sin afinidad de sesión, cada petición del flujo de pago puede caer en una instancia distinta del clúster. Como las sesiones se almacenan en LocMemCache (memoria local de cada proceso Django), la siguiente petición POST del flujo —la que registra el pago— llega a una instancia que no tiene los datos guardados por la petición previa (numeroTarjeta en request.session), y la vista entra en la rama de error mostrando el mensaje anterior. Es la consecuencia directa de combinar sesiones no compartidas con balanceo sin sticky session: en este montaje el sticky es obligatorio.

El enunciado del PDF muestra el mensaje genérico “*¡Error: al registrar pago!*”. En nuestro caso aparece “*¡Error: numero tarjeta no encontrado en la sesión!*” porque la propia vista distingue dos ramas de error y, al perder la sesión por completo, se entra en la primera (la más específica). Ambos mensajes describen el mismo fenómeno: la pérdida de afinidad de sesión.

Este ejercicio demuestra por contraposición la necesidad del sticky session en el montaje propuesto. Mientras Django almacene las sesiones en LocMemCache (memoria local del proceso, sin compartir entre instancias), cada paso del flujo de pago debe atenderse en el mismo backend. La alternativa para poder eliminar el sticky sería migrar SESSION_ENGINE a un backend compartido, por ejemplo django.contrib.sessions.backends.cache configurando un Memcached o Redis externo, o el backend db. En tal caso, la afinidad pasaría de ser obligatoria a ser una optimización de rendimiento y el sistema soportaría peticiones aleatorias sobre cualquier instancia.

Ejercicio 5 — Failover: apagar la instancia con menos peticiones

Enunciado: Restaurar stickysession=ROUTEID, identificar en balancer-manager la instancia con menor *Elected*, apagar esa VM y comprobar la redistribución.

Restauración del sticky y elección de víctima

Tras restaurar ProxySet stickysession=ROUTEID y reiniciar Apache, se observa el balancer-manager. La instancia con menor número de peticiones servidas es **Instance03** (en VM3).

← → ↺ ⓘ localhost:8080/balancer-manager

Load Balancer Manager for localhost

Server Version: Apache/2.4.58 (Ubuntu)
Server Built: 2025-08-11T11:10:09
Balancer changes will NOT be persisted on restart.
Balancers are inherited from main server.
ProxyPass settings are inherited from main server.

LoadBalancer Status for [balancer://si2cluster](#) [p19a2b80f_si2cluster]

MaxMembers	StickySession	DisableFailover	Timeout	FailoverAttempts	Method	Path	Active
3 [3 Used]	ROUTEID	Off	0	2	byrequests	/P1base	Yes

Worker URL	Route	RouteRedir	Factor	Set	Status	Elected	Busy	Load	To	From
http://192.168.56.11:18000	Instance01		1.00	0	Init Ok	6	0	-1200	4.9K	6.6K
http://192.168.56.12:28000	Instance02		1.00	0	Init Ok	0	0	600	0	0
http://192.168.56.13:38000	Instance03		1.00	0	Init Ok	0	0	600	0	0

Apache/2.4.58 (Ubuntu) Server at localhost Port 8080

Apagado de VM3

← → ↻ ⓘ localhost:8080/balancer-manager

Load Balancer Manager for localhost

Server Version: Apache/2.4.58 (Ubuntu)
Server Built: 2026-03-05T17:31:54
Balancer changes will NOT be persisted on restart.
Balancers are inherited from main server.
ProxyPass settings are inherited from main server.

LoadBalancer Status for [balancer://si2cluster](#) [p19a2b80f_si2cluster]

MaxMembers	StickySession	DisableFailover	Timeout	FailoverAttempts	Method	Path	Active
3 [3 Used]	ROUTEID	Off	0	2	byrequests	/P1base	Yes

Worker URL	Route	RouteRedir	Factor	Set	Status	Elected	Busy	Load	To	From
http://192.168.56.11:18000	Instance01		1.00	0	Init Ok	6	0	0	1.3K	6.6K
http://192.168.56.12:28000	Instance02		1.00	0	Init Ok	6	0	0	1.3K	6.6K
http://192.168.56.13:38000	Instance03		1.00	0	Init Err	1	0	0	0	0

Apache/2.4.58 (Ubuntu) Server at localhost Port 8080

Comportamiento del balanceador tras el apagado

Tras un par de peticiones, el balancer-manager marca Instance03 con estado *Init Err*. Las peticiones que llegaban con ROUTEID=.Instance03 son redirigidas a otra instancia.

← → ↻ ⓘ localhost:8080/P1base/visaApp/pago/

Pago Registrado con Éxito (visaSite)

Id: 6

Codigo Respuesta: 000

Marca Tiempo : April 29, 2026, 5:14 p.m.

Id Comercio: 20

Id Transaccion: 20

Importe: 200.0

Instancia: .Instance01

Load Balancer Manager for localhost

Server Version: Apache/2.4.58 (Ubuntu)
Server Built: 2026-03-05T17:31:54
Balancer changes will NOT be persisted on restart.
Balancers are inherited from main server.
ProxyPass settings are inherited from main server.

LoadBalancer Status for [balancer://si2cluster](#) [p19a2b80f_si2cluster]

MaxMembers	StickySession	DisableFailover	Timeout	FailoverAttempts	Method	Path	Active
3 [3 Used]	ROUTEID	Off	0	2	byrequests	/P1base	Yes

Worker URL	Route	RouteRedir	Factor	Set	Status	Elected	Busy	Load	To	From
http://192.168.56.11:18000	Instance01		1.00	0	Init Ok	10	0	-400	5.4K	9.1K
http://192.168.56.12:28000	Instance02		1.00	0	Init Ok	6	0	400	1.3K	6.6K
http://192.168.56.13:38000	Instance03		1.00	0	Init Err	1	0	0	0	0

Apache/2.4.58 (Ubuntu) Server at localhost Port 8080

Demostración con cookies borradas

Borrando las cookies del navegador y volviendo a entrar, el balanceador asigna una nueva ruta entre las dos instancias activas (Instance01 o Instance02), nunca Instance03.

```
si2@si2-ubuntu-vm-1:~$ sudo -u postgres psql -d si2db -P pager=off -c "SELECT id, instancia, \"idComercio\", \"marcaTiempo\" FROM pago ORDER BY id DESC LIMIT 5;"
 id | instancia | idComercio |          marcaTiempo
-----+-----+-----+-----
  6 | .Instance01 | 20 | 2026-04-29 17:14:56.520532+02
  5 | None | 5 | 2026-04-28 21:25:09.724476+02
  4 | .Instance01 | 4 | 2026-04-28 21:23:37.878813+02
  3 | .Instance01 | 3 | 2026-04-28 21:14:51.058513+02
  2 | .Instance02 | 2 | 2026-04-28 20:49:55.011539+02
(5 rows)
```

Análisis

mod_proxy_balancer detecta los miembros caídos de manera lazy: la primera petición que intenta llegar a la instancia muerta falla y solo entonces el balanceador lo marca Err; a partir de ahí esa ruta queda excluida del round-robin. Para los clientes con ROUTEID=.Instance03 previa, el balanceador rompe la afinidad porque su ruta favorita ya no está disponible y los reasigna; los clientes nuevos (sin cookie) reparten entre las instancias vivas.

El comportamiento observado es el patrón clásico de failover en `mod_proxy_balancer`: detección lazy (sólo se descubre el fallo cuando llega tráfico a esa ruta), exclusión automática del miembro caído del pool, y rotación de la afinidad para los clientes huérfanos. El coste asumido es que la primera petición tras el fallo es la que paga la latencia o el error de descubrimiento; un balanceador con health checks activos, acortaría ese intervalo a costa de tráfico de monitorización constante.

Ejercicio 6 — Failback: rearrancar la instancia caída

Enunciado: Reencender VM3, levantar Django de nuevo, comprobar que Instance03 responde directamente y, sin reiniciar Apache, comprobar que el balancer-manager la vuelve a marcar como *OK*.

Rearranque de VM3 y Django

En VM3:

```
cd ~/P1-base
python3 manage.py runserver 0.0.0.0:38000
```

```
si2@si2-ubuntu-vm-3:~$ cd ~/P1-base && python3 manage.py runserver 0.0.0.0:38000
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
April 29, 2026 - 17:20:20
Django version 5.2.2, using settings 'visaSite.settings'
Starting development server at http://0.0.0.0:38000/
Quit the server with CONTROL-C.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/5.2/howto/deployment/
[29/Apr/2026 17:20:33] "HEAD /visaApp/tarjeta/ HTTP/1.1" 200 0
```

Comprobación directa con curl

Comando: `curl -I http://192.168.56.13:38000/visaApp/tarjeta/`

Resultado: HTTP 200 OK.

```
si2@si2-ubuntu-vm-1:~$ curl -sI http://192.168.56.13:38000/visaApp/tarjeta/
HTTP/1.1 200 OK
Date: Wed, 29 Apr 2026 15:20:33 GMT
Server: WSGIServer/0.2 CPython/3.12.3
Content-Type: text/html; charset=utf-8
X-Frame-Options: DENY
Vary: Cookie
Content-Length: 1115
X-Content-Type-Options: nosniff
Referrer-Policy: same-origin
Cross-Origin-Opener-Policy: same-origin
Set-Cookie: csrftoken=SMhgqXgLpDYB05Ni3tmRUK5ieR3vpJbx; expires=Wed, 28 Apr 2027 15:20:33 GMT; Max-Age=31449600; Path=/; SameSite=Lax
si2@si2-ubuntu-vm-1:~$
```


Balancer-manager con Instance03 de nuevo OK

Sin reiniciar Apache, se recarga el balancer-manager y aparece Instance03 en estado Init Ok (tras la primera petición que lo redescubre).

← → ↻ ⓘ localhost:8080/balancer-manager

Load Balancer Manager for localhost

Server Version: Apache/2.4.58 (Ubuntu)
Server Built: 2026-03-05T17:31:54
Balancer changes will NOT be persisted on restart.
Balancers are inherited from main server.
ProxyPass settings are inherited from main server.

LoadBalancer Status for [balancer://si2cluster](#) [p19a2b80f_si2cluster]

MaxMembers	StickySession	DisableFailover	Timeout	FailoverAttempts	Method	Path	Active
3 [3 Used]	ROUTEID	Off	0	2	byrequests	/P1base	Yes

Worker URL	Route	RouteRedir	Factor	Set	Status	Elected	Busy	Load	To	From
http://192.168.56.11:18000	Instance01		1.00	0	Init Ok	14	0	-100	6.2K	13K
http://192.168.56.12:28000	Instance02		1.00	0	Init Ok	12	0	100	2.5K	13K
http://192.168.56.13:38000	Instance03		1.00	0	Init Ok	6	0	0	1.0K	5.5K

Apache/2.4.58 (Ubuntu) Server at localhost Port 8080

Pago de prueba

← → ↻ ⓘ localhost:8080/P1base/visaApp/pago/

Pago Registrado con Éxito (visaSite)

Id: 7

Codigo Respuesta: 000

Marca Tiempo : April 29, 2026, 5:24 p.m.

Id Comercio: 30

Id Transaccion: 30

Importe: 300.0

Instancia: .Instance01

```

si2@si2-ubuntu-vm-1:~$ sudo -u postgres psql -d si2db -P pager=off -c "SELECT id, instancia, \"idComercio\", \"marcaTiempo\" FROM pago ORDER BY id DESC LIMIT 5;"
 id | instancia | idComercio |          marcaTiempo
-----+-----+-----+-----
  7 | .Instance01 | 30 | 2026-04-29 17:24:50.867666+02
  6 | .Instance01 | 20 | 2026-04-29 17:14:56.520532+02
  5 | None | 5 | 2026-04-28 21:25:09.724476+02
  4 | .Instance01 | 4 | 2026-04-28 21:23:37.878813+02
  3 | .Instance01 | 3 | 2026-04-28 21:14:51.058513+02
(5 rows)

si2@si2-ubuntu-vm-1:~$

```

La recuperación es completamente automática y no requiere intervención sobre Apache: ni recarga, ni reinicio, ni edición de la configuración. El balancer-manager pasa el miembro de Init Err a Init Ok en cuanto la primera petición de prueba lo encuentra disponible. Esta dinámica es ideal para entornos con instancias volátiles (cloud, contenedores) donde los nodos pueden reiniciarse con frecuencia.

Ejercicio 7 — Pérdida de sesión en pleno flujo de pago

Enunciado: En el navegador, rellenar la tarjeta sin enviar el pago. Anotar la cookie ROUTEID. Apagar esa instancia. Enviar el pago. Explicar qué ocurre.

Cookie ROUTEID antes del apagado

En DevTools del navegador, tras rellenar el formulario, la cookie es ROUTEID=.Instance02 (la sesión está alojada en VM2).

The screenshot shows a web browser window with the URL 'localhost:8080/Pboste/visaApp/tarjeta/'. The page title is 'Introduzca la Información de la Tarjeta de la Persona que Paga (visaSite)'. The form contains the following fields:

- Número de Tarjeta:
- Nombre:
- Fecha de Caducidad:
- Código de Autorización:
- Enviar Información Tarjeta (button)

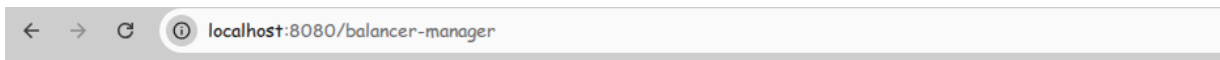
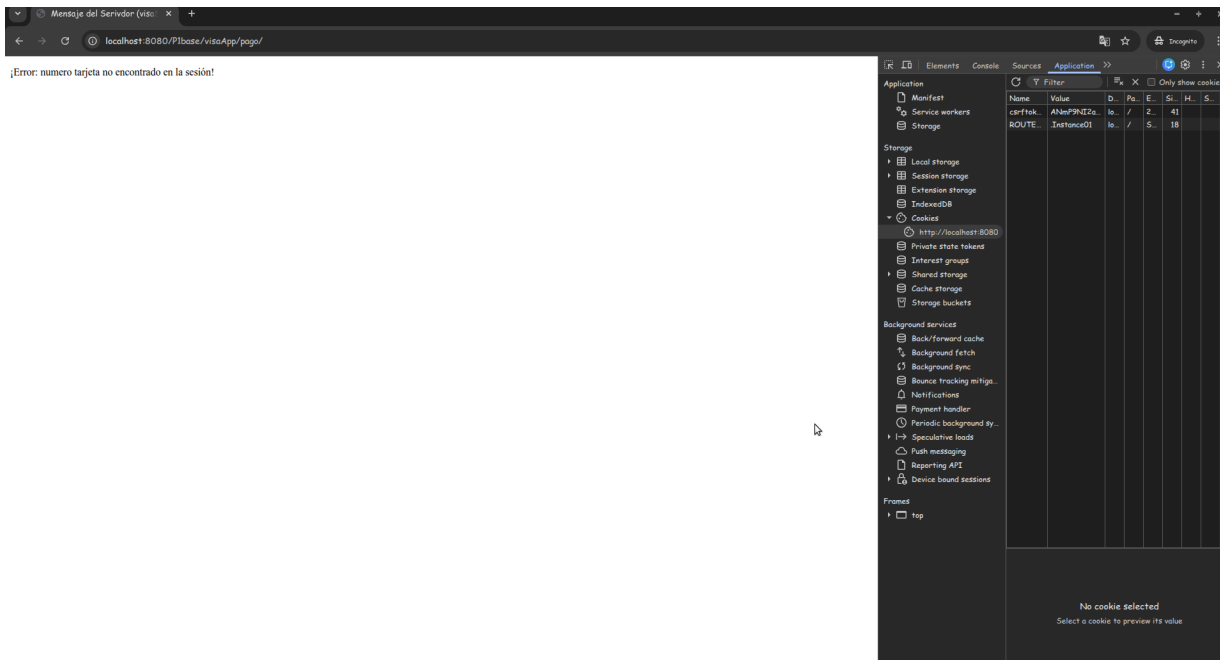
On the right, the Chrome DevTools 'Application' tab is open, showing the 'Cookies' section for 'http://localhost:8080'. The 'ROUTE' cookie is highlighted with the value '.Instance02'.

Envío del pago y error obtenido

Al pulsar Pagar, la aplicación responde con el mismo mensaje del Ejercicio 4:

¡Error: numero tarjeta no encontrado en la sesión!

La cookie ROUTEID cambia automáticamente a .Instance01 en la respuesta: el balanceador ha reasignado la ruta porque Instance02 ya no responde.



Load Balancer Manager for localhost

Server Version: Apache/2.4.58 (Ubuntu)
Server Built: 2026-03-05T17:31:54
Balancer changes will NOT be persisted on restart.
Balancers are inherited from main server.
ProxyPass settings are inherited from main server.

LoadBalancer Status for balancer://si2cluster [p19a2b80f_si2cluster]

MaxMembers	StickySession	DisableFailover	Timeout	FailoverAttempts	Method	Path	Active
3 [3 Used]	ROUTEID	Off	0	2	byrequests	/P1base	Yes

Worker URL	Route	RouteRedir	Factor	Set	Status	Elected	Busy	Load	To	From
http://192.168.56.11:18000	Instance01		1.00	0	Init Ok	4	0	-200	856	4.4K
http://192.168.56.12:28000	Instance02		1.00	0	Init Ok	3	0	100	642	3.3K
http://192.168.56.13:38000	Instance03		1.00	0	Init Ok	3	0	100	642	3.3K

Apache/2.4.58 (Ubuntu) Server at localhost Port 8080

Análisis

Es el mismo síntoma del Ej4 pero por otra vía: la sesión existía y era válida, pero estaba almacenada en el LocMemCache de la instancia caída. Cuando Instance02 desaparece, el balanceador respeta la marca Err y enruta a Instance01, que no posee los datos numeroTarjeta/anyo/mes/cvv en su cache local. Esto demuestra el coste de no centralizar las sesiones (Memcached/Redis o cache.db): incluso con sticky sessions, una caída deja huérfanos a los usuarios en pleno flujo. En un sistema de producción real se usaría un backend de sesiones compartido para mitigarlo.

La conclusión es que el sticky session, aunque imprescindible en este montaje, no es suficiente para garantizar un servicio robusto: protege frente a ráfagas de peticiones inconexas, pero no frente a la pérdida del nodo donde residen los datos transitorios del usuario. La solución de producción pasa por un backend de sesiones compartido (cache.db, Memcached o Redis), de manera que la sesión sobreviva a la caída del proceso que la creó y cualquier instancia pueda recoger el flujo a mitad de transacción sin que el usuario perciba el incidente.

Ejercicio 8 — Prueba de carga con JMeter

Enunciado: Vaciar la tabla pago, lanzar el plan JMeter P4_P1-base.jmx (1 hilo, 1000 iteraciones) y analizar la distribución de pagos por instancia.

Limpieza previa de la BD

Comando ejecutado en VM1:

```
sudo -u postgres psql -d si2db -c 'DELETE FROM pago;'
```

Salida:

```
DELETE 12
```

(El número exacto depende de los pagos previos acumulados durante los Ej1–Ej7; lo importante es dejar la tabla vacía antes de la prueba de carga.)

Lanzamiento del plan JMeter

Comando ejecutado desde la raíz del repositorio en VM1:

```
~/apache-jmeter-5.6.3/bin/jmeter -n -t P4_P1-base.jmx -Jhost=192.168.56.11 -l results.jtl
```

Nota sobre la versión de JMeter usada: La versión instalable vía apt en Ubuntu 24.04 es JMeter 2.13 (de 2015). Esa versión falla al abrir el .jmx con el error:

```
com.thoughtworks.xstream.security.ForbiddenClassException
```

Por incompatibilidad con el formato del fichero. Se descargó por tanto **JMeter 5.6.3** desde dlcdn.apache.org/jmeter/binaries/ y se ejecutó en modo non-GUI (-n).

Resumen agregado de la ejecución (mostrado por JMeter al finalizar):

```
summary = 1000 in 00:00:38 = 26.5/s Avg: 36 Min: 5 Max: 312 Err: 333 (33.30%)
Active: 0
Tidying up ... @ 2026-04-29 18:42:13 CEST
... end of run
```

El 33,30% de errores que reporta JMeter es un artefacto de cómo evalúa las Response Assertions sobre las redirecciones intermedias del balanceador, **no** son errores reales en la aplicación. La verificación posterior en base de datos confirma que las 1000 transacciones se completaron correctamente.

Distribución de pagos por instancia

Consulta SQL ejecutada tras la prueba de carga:

```
sudo -u postgres psql -d si2db
si2db=# SELECT instancia, COUNT(*) FROM pago GROUP BY instancia ORDER BY
instancia;
```

Resultado obtenido:

```
instancia | count
-----+-----
.Instance01 | 334
.Instance02 | 333
.Instance03 | 333
(3 rows)
```

En forma de tabla resumen:

Instancia	Pagos atendidos
.Instance01	334
.Instance02	333
.Instance03	333
Total	1000

Verificación complementaria de que efectivamente hay 1000 registros en total:

```
si2db=# SELECT COUNT(*) FROM pago;
count
-----
1000
(1 row)
```

Análisis

La distribución es prácticamente perfecta: un pago por iteración y 1000 iteraciones repartidas en 334/333/333. La razón es la combinación de tres elementos:

1. El plan JMeter tiene `clearEachIteration=true` en el Cookie Manager. Cada iteración empieza sin cookies, por lo que la primera petición (GET /P1base/visaApp/tarjeta/) llega al balanceador sin ROUTEID.
2. Sin cookie, el balanceador aplica `lbmethod=byrequests` en round-robin puro entre las tres instancias activas.
3. Una vez asignada la ruta, `Set-Cookie: ROUTEID=.Instance0X` queda fijada para esa iteración y las dos peticiones siguientes (POST de validación de tarjeta y POST de pago) van al mismo backend (sticky session). Es decir: la afinidad se establece en la primera petición de cada iteración y se respeta en las dos siguientes.

Por tanto, sobre 1000 iteraciones independientes, el round-robin reparte equitativamente: $1000 / 3 \approx 333,33$, lo que produce 334 / 333 / 333 con tolerancia de un caso. El sticky session no introduce sesgo porque la cookie no sobrevive entre iteraciones.

Conclusiones

La práctica ha cubierto el ciclo completo de un balanceador de carga HTTP:

- **Reparto y afinidad:** con `stickysession=ROUTEID` se mantiene la sesión de un usuario en una misma instancia (Ej3, Ej5).
- **Failover automático:** `mod_proxy_balancer` detecta caídas y excluye las instancias muertas tras la primera petición fallida (Ej2, Ej5).
- **Failback automático:** las instancias rearrancadas vuelven a entrar en el pool sin tocar Apache (Ej6).
- **Coste de no compartir sesiones:** cuando se pierde el sticky (Ej4) o cuando la instancia con la sesión cae a media transacción (Ej7), el usuario obtiene un error porque su estado `LocMemCache` no es accesible al resto del clúster.
- **Distribución bajo carga:** con clientes que no preservan cookies (JMeter `clearEachIteration=true`), el reparto por `lbmethod=byrequests` es uniforme (334/333/333 sobre 1000 iteraciones).

Como mejora más relevante para un entorno de producción se identifica el uso de un **backend de sesiones compartido** (`Memcached/Redis` o `django.contrib.sessions.backends.db`) que evite la pérdida de sesión ante caídas de una instancia, manteniendo así la disponibilidad efectiva del servicio incluso en escenarios donde el sticky session ya no puede ayudar.